

1. Objectives:

1. To understand the basic concept of a Circular Linked List.
2. To learn the operations of insertion at the beginning, at the end, deletion of the first node, last node, and traversal.

2. Introduction:

- **Circular Singly Linked List:** Circular Singly Linked List is a special type of linked list where each node contains a value and a pointer to the next node, but unlike a normal singly linked list, the last node does not point to NULL. Instead, it points back to the first node, forming a circular structure. The first node is referenced by a pointer called start or head. In this structure, each node consists of two parts: data and a pointer to the next node. This circular connection allows traversal to continue repeatedly from the last node to the first node without interruption.

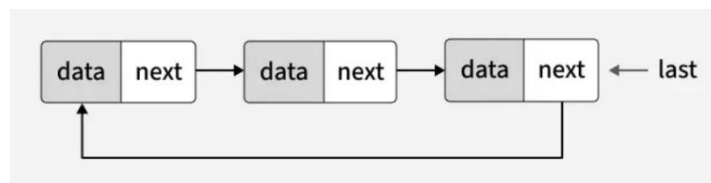


Fig 7.1: Circular Linked List

To insert a new node at the beginning, a node is created and its next pointer is set to the current head. Then the last node is updated so that its next pointer points to the new node, and finally the head is updated to the new node. For insertion at the end, the list is traversed until the last node, and the new node is added by updating the last node's next pointer, while the new node's next pointer is set to the head to maintain the circular structure.

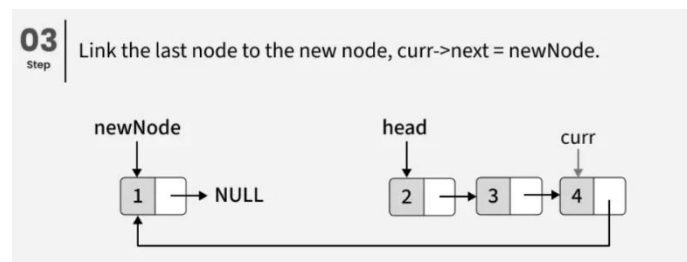


Fig 7.2: Circular Linked List Insertion

To delete a node, careful pointer adjustment is required to maintain the circular link. For deleting the first node, the last node's next pointer is updated to point to the second node, and the head is moved forward. For deleting the last node, the list is traversed to the second last node, and its next pointer is updated to point to the head, removing the last node while preserving the circular connection.

3. Procedure

Experiment: Circular Linked List

Input:

- k – Integer value to be inserted into the linked list
- start – Pointer to the first node of the list

Output:

- Printed values of all nodes in the linked list

Algorithm:**InsertStart(start, k):**

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. $\text{start} \leftarrow n$
4. $n \rightarrow \text{next} \leftarrow \text{start}$
5. **return**
6. $\text{temp} \leftarrow \text{start}$
7. **while** temp->next != start **do**
8. $\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$
9. $n \rightarrow \text{next} \leftarrow \text{start}$
10. $\text{temp} \rightarrow \text{next} \leftarrow n$
11. $\text{start} \leftarrow n$

InsertEnd(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. $\text{start} \leftarrow n$
4. $n \rightarrow \text{next} \leftarrow \text{start}$
5. **return**
6. $\text{temp} \leftarrow \text{start}$
7. **while** temp->next != start **do**
8. $\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$
9. $\text{temp} \rightarrow \text{next} \leftarrow n$
10. $n \rightarrow \text{next} \leftarrow \text{start}$

DeleteFirst(start):

1. $\text{temp} \leftarrow \text{start}$
2. **while** temp->next != start **do**
3. $\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$
4. $\text{start} \leftarrow \text{start} \rightarrow \text{next}$
5. $\text{temp} \rightarrow \text{next} \leftarrow \text{start}$

DeleteLast(start):

1. $\text{temp} \leftarrow \text{start}$
2. **while** temp->next->next != start **do**
3. $\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$
4. $\text{temp} \rightarrow \text{next} \leftarrow \text{start}$

Traverse(start):

1. $\text{temp} \leftarrow \text{start}$
2. **while** temp->next != start **do**
3. $\text{print temp} \rightarrow \text{val}$
4. $\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$
5. **print temp} \rightarrow \text{val}**

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment: Circular Linked List

```
#include <bits/stdc++.h>
using namespace std;

class Node{
public:
    int val;
    Node *next;
    Node(int k){
        val = k;
        next = nullptr;
    }
};

class List{
public:

    Node *start = nullptr;

    void insFirst(int k){
        Node *n = new Node(k);
        if (start == nullptr){
            start = n;
            n->next = start;
            return;
        }

        Node *temp = start;
        while (temp->next != start){
            temp = temp->next;
        }
        n->next = start;
        temp->next = n;
        start = n;
    }

    void insLast(int k){
        Node *n = new Node(k);
        if (start == nullptr){
            start = n;
            n->next = start;
            return;
        }

        Node *temp = start;
        while (temp->next != start){
            temp = temp->next;
        }
        temp->next = n;
        n->next = start;
    }

    void deleteFirst(){
        Node *temp = start;
        while (temp->next != start){
            temp = temp->next;
        }
        start = start->next;
        temp->next = start;
    }

    void deleteLast(){
        Node *temp = start;
        while (temp->next->next != start){
            temp = temp->next;
        }
        temp->next = start;
    }
}
```

```

void traverse(){
    Node *temp = start;
    while (temp->next != start){
        cout << temp->val << "\t";
        temp = temp->next;
    }
    cout << temp->val << endl;
}

};

int main(){
    List l;
    l.insFirst(2);
    l.insFirst(4);
    l.insFirst(1);
    l.traverse();

    l.insLast(3);
    l.insLast(5);
    l.insLast(9);
    l.traverse();

    l.deleteFirst();
    l.traverse();

    l.deleteLast();
    l.traverse();

    return 0;
}

```

Table 7.1: Experiment 1 output

Input	Expected Output						Actual Output					
	1	4	2				1	4	2			
	1	4	2	3	5	9	1	4	2	3	5	9
	4	2	3	5	9		4	2	3	5	9	
	4	2	3	5			4	2	3	5		

6. Discussion & Conclusion:

In this lab we successfully implemented a Circular Singly Linked List and performed various operations including insertion at the beginning, insertion at the end, deletion of the first node, deletion of the last node, and traversal of the list. We learned that maintaining the circular structure we have to be careful about updating pointers, especially about the last node which always points back to the start node. Compared to a normal singly linked list, circular linked lists do not have a NULL pointer at the end, which allows continuous traversal. Overall, this experiment helped in understanding circular data structures and improved our ability to manage pointers and dynamic memory efficiently.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/circular-linked-list/> accessed at 6.25 PM on 18th April 2026.